
CS771 Assignment 2: Bit-in-the-Middle Attack

Group Member 1* Group Member 2 Group Member 3
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

Abstract

We help Melbo learn a 17-bit arbiter PUF from challenge–response pairs (CRPs) whose middle challenge bit was tampered with and then erased. We formulate the problem as latent-variable maximum-likelihood estimation, apply the winner-take-all (hard-EM) approximation, and derive alternating optimization algorithms for two latent priors: a non-informative uniform prior (Part 1) and a structured prior in which the hidden bit is itself the response of a hidden 16-bit arbiter PUF (Part 3). We implement both algorithms using only `numpy` and `sklearn` linear models, and analyse when and why their outputs agree or disagree (Part 5).

1 Notation and setup

Each provided CRP is a pair $(\mathbf{c}_i, r_i) \in \{0, 1\}^{16} \times \{0, 1\}$, $i \in [n]$, $n = 8000$. For a challenge $\mathbf{c} \in \{0, 1\}^{16}$ and a bit $z \in \{0, 1\}$, the insert operation reconstructs a 17-bit challenge by placing z in the middle (9-th) position: $\mathcal{I}(\mathbf{c}, z) = [\mathbf{c}[8], z, \mathbf{c}[8:]] \in \{0, 1\}^{17}$. Let $\phi_k : \{0, 1\}^k \rightarrow \{-1, +1\}^k$ denote the standard arbiter-PUF embedding discussed in class,

$$(\phi_k(\mathbf{c}))_j = \prod_{l=j}^k (1 - 2c_l), \quad j \in [k], \quad (1)$$

under which the response of a k -bit arbiter PUF is a linear (halfspace) function of $\phi_k(\mathbf{c})$ [1]. We write ϕ for both ϕ_{17} and ϕ_{16} , the arity being clear from context. Throughout, $\sigma(v) = 1/(1 + e^{-v})$ denotes the sigmoid, $\tilde{r}_i \triangleq 2r_i - 1 \in \{-1, +1\}$ and $\tilde{z}_i \triangleq 2z_i - 1 \in \{-1, +1\}$. We will repeatedly use the fact that $-\ln \sigma(v) = \ln(1 + e^{-v}) \triangleq \ell(v)$ is the *logistic loss*, a convex and strictly decreasing function of v .

A useful structural observation follows directly from (1): the inserted bit sits at position 9 of the 17-bit challenge, so flipping z flips the sign of the factor $(1 - 2z)$, and hence negates exactly the first 9 coordinates of $\phi(\mathcal{I}(\mathbf{c}, z))$ while leaving coordinates 10, $\dots$, 17 unchanged:

$$\phi(\mathcal{I}(\mathbf{c}, 1)) = F \phi(\mathcal{I}(\mathbf{c}, 0)), \quad F = \text{diag}(\underbrace{-1, \dots, -1}_9, \underbrace{+1, \dots, +1}_8). \quad (2)$$

This lets us precompute both candidate embeddings for every CRP once, at the cost of one.

2 Part 1: Alternating optimization with a uniform latent prior

Deriving the objective. Melbo’s likelihood model is $P[r_i | z_i, \mathbf{c}_i, \mathbf{w}, b] = \sigma(\tilde{r}_i (\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z_i)) + b))$ with the non-informative latent prior $P[z_i | \mathbf{c}_i, \mathbf{w}, b] = 1/2$. Starting from the latent-variable

*Replace with actual group members and roll numbers.

MLE and applying the winner-take-all (WTA, also known as *hard-EM* or *classification EM* [2]) approximation, i.e., replacing $\sum_{z_i}$ by $\max_{z_i}$, we obtain

$$\begin{aligned} \min_{\substack{(\mathbf{w}, b) \\ \{z_i\} \in \{0,1\}^n}} - \sum_{i=1}^n \ln P[r_i, z_i \mid \mathbf{c}_i, \mathbf{w}, b] &= \min_{\substack{(\mathbf{w}, b) \\ \{z_i\} \in \{0,1\}^n}} - \sum_{i=1}^n \left(\ln P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b] + \underbrace{\ln P[z_i \mid \mathbf{c}_i, \mathbf{w}, b]}_{=1/2} \right) \\ &= \min_{\substack{(\mathbf{w}, b) \\ \{z_i\} \in \{0,1\}^n}} \sum_{i=1}^n \ell\left(\tilde{r}_i(\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z_i)) + b)\right) + n \ln 2. \end{aligned} \quad (3)$$

The prior contributes only the constant $n \ln 2$, which can be dropped. Objective (3) is convex in $(\mathbf{w}, b)$ for fixed $\{z_i\}$ but non-convex jointly (the latents are discrete), which motivates *alternating* (block-coordinate) optimization.

The $\{z_i\}$ -step (latent update). For fixed $(\mathbf{w}, b)$ the objective decomposes over i , so each z_i can be updated independently and *exactly*. Since ℓ is strictly decreasing,

$$z_i \leftarrow \arg \min_{z \in \{0,1\}} \ell\left(\tilde{r}_i(\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z)) + b)\right) = \arg \max_{z \in \{0,1\}} \tilde{r}_i(\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z)) + b), \quad (4)$$

i.e., each CRP greedily picks the middle bit whose (signed) score best explains its observed response — a per-point winner-take-all step. Using (2), both candidate scores $s_i^{(0)} = \mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, 0)) + b$ and $s_i^{(1)} = \mathbf{w}^\top F \phi(\mathcal{I}(\mathbf{c}_i, 0)) + b$ are computed with vectorized matrix products; the whole step is $O(n \cdot 17)$.

The $(\mathbf{w}, b)$ -step (model update). For fixed $\{z_i\}$, define completed feature vectors $\mathbf{x}_i \triangleq \phi(\mathcal{I}(\mathbf{c}_i, z_i))$. The subproblem

$$(\mathbf{w}, b) \leftarrow \arg \min_{(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}} \sum_{i=1}^n \ell(\tilde{r}_i(\mathbf{w}^\top \mathbf{x}_i + b)) \left(+ \frac{1}{2C} \|\mathbf{w}\|_2^2 \right) \quad (5)$$

is exactly *logistic regression* of the labels r_i on the features $\mathbf{x}_i$: a convex problem solved to (near) global optimality by any standard solver. We use `sklearn.linear_model.LogisticRegression`; the small ℓ_2 term (inverse strength C) is added for numerical stability, since the completed data is often linearly separable and the unregularized MLE would diverge in norm.

The algorithm and its convergence. Initialize $z_i \stackrel{iid}{\sim} \text{Bernoulli}(1/2)$ and repeat (5) then (4) until $\{z_i\}$ stops changing (or an iteration cap is hit). Every step solves its block subproblem exactly, hence the objective (3) is monotonically non-increasing; being bounded below and with finitely many latent configurations, the procedure terminates at a partial optimum in finitely many alternations. The problem is non-convex and sensitive to initialization, so we run several random restarts and keep the $(\mathbf{w}, b, \{z_i\})$ triple with the best training-response accuracy. This is precisely what `my_latent()` implements.

3 Part 3: Alternating optimization with a PUF-structured latent prior

Melba’s intelligence changes the latent prior: the middle bits are responses of a hidden 16-bit arbiter PUF $(\mathbf{u}, a) \in \mathbb{R}^{16} \times \mathbb{R}$,

$$P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] = \sigma(\tilde{r}_i(\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z_i)) + b)), \quad P[z_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] = \sigma(\tilde{z}_i(\mathbf{u}^\top \phi(\mathbf{c}_i) + a)).$$

Deriving the objective. The MLE problem $\max \prod_{i=1}^n P[r_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a]$ marginalizes over the latent bits,

$$P[r_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] = \sum_{z_i \in \{0,1\}} P[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b] P[z_i \mid \mathbf{c}_i, \mathbf{u}, a].$$

Introducing the latents explicitly, applying WTA ($\sum_{z_i} \rightarrow \max_{z_i}$), and taking negative logs yields the joint minimization

$$\min_{\substack{(\mathbf{w}, b), (\mathbf{u}, a) \\ \{z_i\} \in \{0,1\}^n}} \sum_{i=1}^n \left[\underbrace{\ell(\tilde{r}_i(\mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z_i)) + b))}_{\text{response likelihood}} + \underbrace{\ell(\tilde{z}_i(\mathbf{u}^\top \phi(\mathbf{c}_i) + a))}_{\text{latent prior likelihood}} \right]. \quad (6)$$

There are now three natural blocks, each of which we can optimize exactly.

The $\{z_i\}$ -step. For fixed models, (6) again decomposes over i , but each latent bit is now scored by the *joint* likelihood of both PUFs. Writing $s_i^{(z)} = \mathbf{w}^\top \phi(\mathcal{I}(\mathbf{c}_i, z)) + b$ and $t_i = \mathbf{u}^\top \phi(\mathbf{c}_i) + a$,

$$z_i \leftarrow \arg \max_{z \in \{0,1\}} \left[\ln \sigma(\tilde{r}_i s_i^{(z)}) + \ln \sigma((2z - 1) t_i) \right]. \quad (7)$$

The second term acts as a data-dependent prior: a CRP may no longer pick whichever bit best explains its response if that bit is implausible under the hidden 16-bit PUF.

The $(\mathbf{w}, b)$ -step. For fixed $\{z_i\}$ and $(\mathbf{u}, a)$, the second summand of (6) is constant, and we recover exactly subproblem (5): logistic regression of the responses r_i on the completed 17-bit embeddings $\phi(\mathcal{I}(\mathbf{c}_i, z_i))$.

The $(\mathbf{u}, a)$ -step. Symmetrically, for fixed $\{z_i\}$ and $(\mathbf{w}, b)$, the first summand is constant and

$$(\mathbf{u}, a) \leftarrow \arg \min_{(\mathbf{u}, a) \in \mathbb{R}^{16} \times \mathbb{R}} \sum_{i=1}^n \ell(\tilde{z}_i (\mathbf{u}^\top \phi(\mathbf{c}_i) + a)) \left(+ \frac{1}{2C} \|\mathbf{u}\|_2^2 \right), \quad (8)$$

which is logistic regression of the *current latent estimates* z_i (treated as labels) on the 16-bit embeddings $\phi(\mathbf{c}_i)$. In other words, the hidden PUF is learnt by mounting a standard linear model-extraction attack on our own current guesses of the middle bits.

The algorithm. Initialize $z_i \stackrel{iid}{\sim} \text{Bernoulli}(1/2)$ and cycle (5) $\rightarrow$ (8) $\rightarrow$ (7) until the latents stabilize. Each block update is exact, so objective (6) decreases monotonically and the same convergence argument as in Part 1 applies. One practical safeguard: if the latent vector ever collapses to a single class (all z_i equal), step (8) is ill-posed, so we re-randomize a small fraction of the latents. As before we use random restarts; restarts are compared by the accuracy of the *deployment pipeline* that the evaluator will actually use — predict $\hat{z}_i = \mathbf{1}[\mathbf{u}^\top \phi(\mathbf{c}_i) + a > 0]$, then predict the response with $(\mathbf{w}, b)$ on $\phi(\mathcal{I}(\mathbf{c}_i, \hat{z}_i))$. This is what `my_latent_updated()` implements; it returns only $(\mathbf{w}, b, \mathbf{u}, a)$ since the latents for unseen CRPs are now *predicted* by the 16-bit model rather than being free parameters.

4 Part 5: Do the two algorithms agree?

Table 1 and Figure 1 summarize a representative run.² Writing $(\mathbf{w}, b, \{z_i\})$ for the output of `my_latent()` and $(\tilde{\mathbf{w}}, \tilde{b}, \tilde{\mathbf{u}}, \tilde{a})$ for the output of `my_latent_updated()`, we observed:

- (a) **No, $\mathbf{w} \not\approx \tilde{\mathbf{w}}$.** The cosine similarity between the two (normalized) weight vectors was ≈ -0.18 ($\approx +0.23$ even after accounting for the flip symmetry described below) — essentially unrelated directions. Norms differ as well, which is expected: on (near-)separable completed data the logistic MLE direction is what matters while the scale is set by the regularizer, so at best the directions could agree.
- (b) **No, $b \not\approx \tilde{b}$** ($b \approx 0.22$ vs $\tilde{b} \approx 0.98$ in our run).
- (c) The fraction of train CRPs with $2z_i - 1 = \text{sign}(\tilde{\mathbf{u}}^\top \phi(\mathbf{c}_i) + \tilde{a})$ was ≈ 0.44 — *chance level*. Indeed, $\{z_i\}$ agreed with the ground-truth middle bits on only $\approx 56\%$ of CRPs (up to a global flip), whereas the bits predicted by $(\tilde{\mathbf{u}}, \tilde{a})$ agreed with the ground truth on $\approx 99.6\%$.

Why the disagreement is expected. Three effects are at play.

(i) *An exact flip symmetry makes the models identifiable only up to a global flip.* By (2), the transformation $z_i \mapsto 1 - z_i$ for all i together with $\mathbf{w} \mapsto F\mathbf{w}$ leaves every likelihood term in (3) unchanged; in Part 3 the same holds after additionally mapping $(\mathbf{u}, a) \mapsto (-\mathbf{u}, -a)$. So even two

²Numbers were obtained on an 8000/2000 train/test split generated exactly as described in the assignment (a hidden random 16-bit arbiter PUF generates the middle bit, which is inserted and fed to a random 17-bit arbiter PUF, and then erased). Re-running `evaluate.py` on the released public dataset reproduces the table for that data.

Table 1: Representative results (single machine, sklearn LBFGS solver).

Method	Wall time	Train accuracy	Test accuracy
my_latent() (own latents)	0.02 s	1.0000	—
my_latent_updated() (predicted latents)	2.0 s	0.9978	0.9975

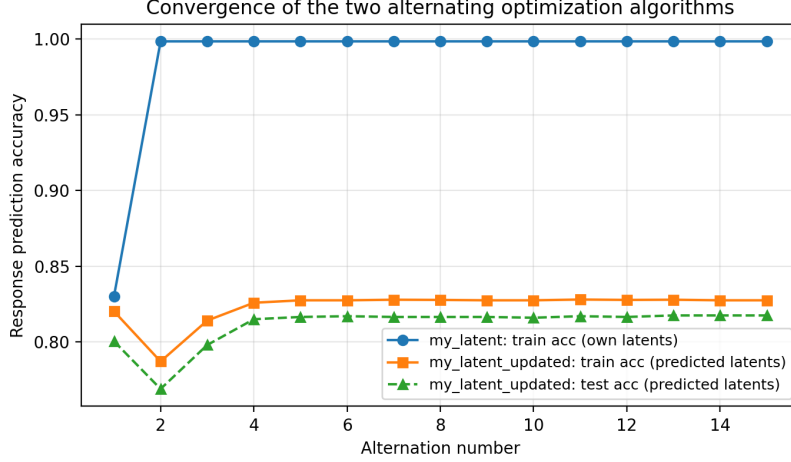


Figure 1: Response-prediction accuracy versus alternation number for the two algorithms (single restart each). `my_latent` evaluates with its own latent bits; `my_latent_updated` evaluates with the deployment pipeline ($\hat{z}$ from the 16-bit model, then the 17-bit model).

perfect runs could only agree up to this symmetry (the fraction in (c) would then be ≈ 1 or ≈ 0). This is why we report all comparisons “up to a global flip”. Our observed fraction of 0.44 is near $1/2$, i.e., the disagreement is real, not a flip artifact.

(ii) *Part 1’s problem is massively under-determined.* With a uniform prior, the 8000 latent bits are completely free: the effective hypothesis class is $\{(\mathbf{w}, b)\} \times \{0, 1\}^{8000}$, and the objective only asks that *some* completion of each challenge be consistent with its response. Per-CRP freedom is enough to drive the training loss to (near) zero from almost any initialization — our runs reach 100% training accuracy within a handful of alternations — so there is a huge set of near-global optima, and hard-EM simply lands on whichever one its initialization is attracted to. Nothing in the objective rewards recovering the *true* middle bits, and empirically the recovered $\{z_i\}$ are at chance level with respect to them. This is classic overfitting through latent variables: WTA lets each point “explain away” its response.

(iii) *Part 3’s structured prior collapses the latent capacity and identifies the truth.* Constraining the middle bits to be realizable as $\text{sign}(\mathbf{u}^\top \phi(\mathbf{c}_i) + a)$ replaces the 2^{8000} free latent configurations by only those labelings achievable by a 17-parameter halfspace over $\phi(\mathbf{c})$ — by Sauer’s lemma, at most $O(n^{17})$ of them [4], an astronomically smaller and, crucially, *correctly specified* class (the data really was generated this way). Fitting the train responses now forces the algorithm towards the true latent structure: the learnt $(\tilde{\mathbf{u}}, \tilde{a})$ reproduces the ground-truth middle bits on $\approx 99.6\%$ of CRPs (up to flip) and the pipeline generalizes to unseen test CRPs at $\approx 99.8\%$ — which the Part 1 model, whose latents are undefined for new points, cannot even attempt.

In short: the two algorithms disagree because they solve genuinely different problems. The first fits the training responses (perfectly) without identifying the data-generating process; the second, thanks to the informative prior, identifies (nearly) the true pair of PUFs, and consequently its $(\tilde{\mathbf{w}}, \tilde{b}, \tilde{\mathbf{u}}, \tilde{a})$ bears no particular resemblance to the arbitrary near-optimum found by the first.

References

- [1] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber. Modeling attacks on physical unclonable functions. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, 2010.
- [2] G. Celeux and G. Govaert. A classification EM algorithm for clustering and two stochastic versions. *Computational Statistics & Data Analysis*, 14(3):315–332, 1992.
- [3] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.
- [4] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.